LifeSynch AI — Student Learning Handbook

Master the Concepts, Architecture, and Code of Your Minor Project

Target Audience: 3rd-Year Undergraduate Students in Artificial Intelligence and Machine Learning (AIML)
Project: LifeSynch AI — Autonomous Academic & Personal Responsibility Hub

---

Table of Contents

1. Welcome & How to Use This Handbook
2. Big Picture: What Does LifeSynch AI Really Do?
3. The Technology Stack Explained in Plain English
   - 3.1 TypeScript — Why Not Plain JavaScript?
   - 3.2 React & Vite — How the UI Works
   - 3.3 Tailwind CSS & Lucide Icons
   - 3.4 Node.js & Express — The Backend Engine
   - 3.5 Supabase & PostgreSQL — The Cloud Database
   - 3.6 Google Gemini AI (LLM) — The Brain
4. Step-by-Step Deep Dives into Core Concepts
   - 4.1 How Google OAuth 2.0 Actually Works
   - 4.2 How Gmail API Fetches Your Emails
   - 4.3 How Google Calendar API Tracks Events
   - 4.4 Automatic Background Synchronization
   - 4.5 AI Responsibility Extraction & Schema Validation
   - 4.6 The "Worth Knowing" Section vs Actionable Tasks
   - 4.7 Deadline & Timezone Intelligence (UTC vs IST)
   - 4.8 Notifications & Reminders Engine
   - 4.9 "Ask LifeSynch AI" Conversational Assistant
   - 4.10 Multi-User Data Isolation & Row Level Security (RLS)
5. Key Code Walkthroughs with Annotations
6. Common Traps & How We Solved Them
7. Glossary of Key Technical Terms

---

1. Welcome & How to Use This Handbook

As an engineering student facing a project viva, you often know *that* your project works, but you might feel nervous when your guide or examiner asks:

- "*Why did you use Row Level Security instead of checking user ID in Express?*"
- "*What happens if Google expires the OAuth token while syncing?*"
- "*How does your AI distinguish between a festival greeting and an exam assignment?*"
- "*Show me exactly where in the database the due time is stored.*"

This handbook was written specifically to give you total confidence. It explains every single mechanism using friendly analogies, visual diagrams, and direct references to our project code.

---

2. Big Picture: What Does LifeSynch AI Really Do?

The Everyday Problem

College students and busy professionals receive dozens of emails every day:

- Assignment announcements with submission deadlines.
- Exam timetables with dates and times.
- Meeting notices, club events, and webinars.
- Fee payment reminders.
- Marketing newsletters and spam.

In addition, their Google Calendar contains lecture schedules, lab exams, and personal events.

Because this information is scattered across Gmail and Calendar, students frequently miss important deadlines or forget to submit project reports on time.

The LifeSynch AI Solution

LifeSynch AI is an **autonomous intelligence layer** between your Google account and your daily schedule:

1. It connects securely to your **Google Account** (OAuth 2.0).
2. It runs a **silent background engine** that fetches new emails and calendar updates every 30 seconds.
3. It passes message content through an **AI extraction pipeline** powered by Google Gemini (with an offline grounded fallback).
4. The AI extracts **actionable responsibilities** (e.g., "Submit Physics Lab Record by Friday 5 PM"), assigns a category (Academic, Work, Personal, Financial), determines priority (High, Medium, Low), and pinpoints the deadline.
5. Non-actionable but important announcements (e.g., "Library will remain closed tomorrow") are categorized into the **"Worth Knowing"** feed so they do not clutter your task list.
6. The system sends **real-time alerts** for overdue items, items due today, and upcoming tasks.
7. You can chat with **"Ask LifeSynch AI"** in natural language to ask questions like: "*What assignments do I have due this week?*" or "*List all my high-priority academic tasks.*"

---

3. The Technology Stack Explained in Plain English

Let us understand each technology used in LifeSynch AI and why we chose it.

3.1 TypeScript — Why Not Plain JavaScript?

- **Plain JavaScript** is dynamically typed. If a function expects `{ title: string, dueDate: string }` and you pass `{ name: "Math", date: 123 }`, JavaScript will not complain until the code crashes in production.
- **TypeScript** adds static types. In LifeSynch AI, shared interfaces like `Responsibility`, `SyncStatus`, and `NotificationItem` are defined once in the `shared/` workspace. Both frontend and backend import the exact same types.
- **Analogy**: Plain JavaScript is like ordering food verbally in a noisy restaurant (high risk of wrong order). TypeScript is like filling out a pre-printed ordering slip where every checkbox is verified before the chef cooks.

3.2 React & Vite — How the UI Works

- **React**: A component-based JavaScript library for building user interfaces. Instead of writing one massive HTML page, we build reusable building blocks (e.g., `<Navbar />`, `<ResponsibilityCard />`, `<QuickStats />`, `<NotificationBell />`).
- **State & Hooks**: When data changes (for example, a new responsibility is added), React's `useState` or custom hooks (like `useResponsibilities`) automatically re-render only that specific part of the screen without refreshing the page.
- **Vite**: A modern frontend build tool that compiles TypeScript and bundles assets in milliseconds using native browser ES modules.

3.3 Tailwind CSS & Lucide Icons

- **Tailwind CSS**: A utility-first CSS framework. Instead of writing separate `.css` files with hundreds of custom classes, we apply utility classes directly on HTML elements (e.g., `flex items-center justify-between p-4 bg-white rounded-xl shadow-sm`). This guarantees a sleek, responsive design for mobile, tablet, and desktop.
- **Lucide React**: Clean, lightweight SVG vector icons (calendar, clock, bell, search, check-circle) that give LifeSynch AI a polished, professional aesthetic.

3.4 Node.js & Express — The Backend Engine

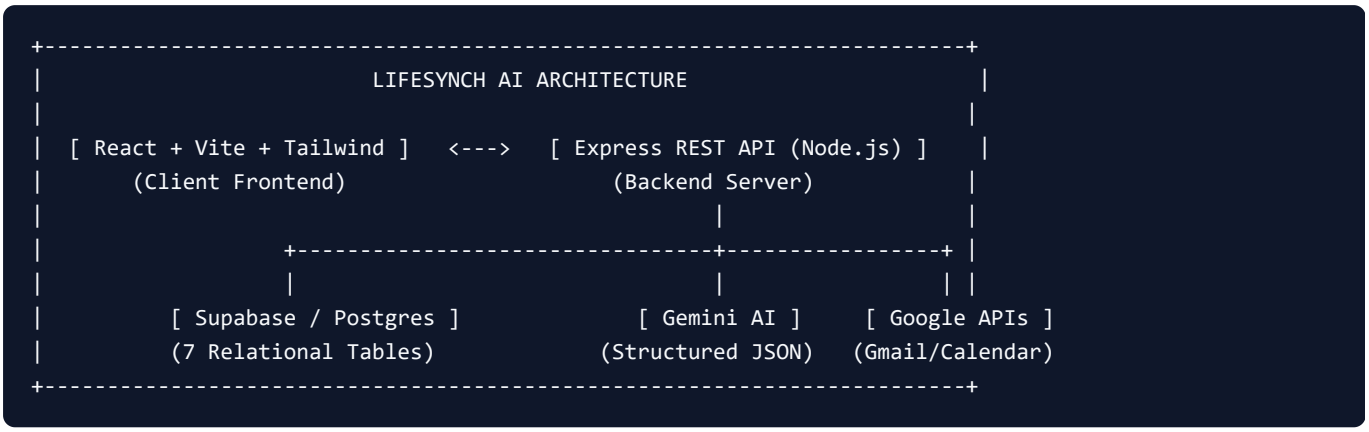
- **Node.js**: Allows us to run JavaScript/TypeScript on the server side using the V8 engine.
- **Express**: A fast, unopinionated web framework for Node.js. It manages our REST API routes:
  - `/api/auth/*` — Handles Google OAuth login, callbacks, and user sessions.
  - `/api/sync/*` — Triggers and reports background sync status.
  - `/api/responsibilities/*` — Lists, filters, updates, and completes tasks.
  - `/api/notifications/*` — Serves due-today, upcoming, and overdue alerts.
  - `/api/assistant/*` — Handles conversational queries.

3.5 Supabase & PostgreSQL — The Cloud Database

- **PostgreSQL**: An enterprise-grade, relational SQL database known for reliability, foreign keys, and JSON support.
- **Supabase**: An open-source Firebase alternative built on PostgreSQL. It provides:
  - Instant REST APIs for tables.
  - Built-in authentication.
  - **Row Level Security (RLS)**: Enforces access control at the database engine level. Even if an attacker knew another user's responsibility ID, PostgreSQL itself rejects the query if `auth.uid() != user_id`.

3.6 Google Gemini AI (LLM) — The Brain

- LifeSynch AI uses Google's state-of-the-art **Gemini Flash** model (`@google/genai` SDK).
- It receives clean email text and calendar summaries, understands the context using Natural Language Understanding (NLU), and outputs structured JSON conforming to our strict **Zod** schema.



---

4. Step-by-Step Deep Dives into Core Concepts

4.1 How Google OAuth 2.0 Actually Works

When a user clicks **"Sign in with Google"**, here is what happens behind the scenes:

1. **User Redirection**: The frontend directs the user's browser to Google's consent screen with requested scopes:
   - `openid` , `email` , `profile`
   - `https://www.googleapis.com/auth/gmail.readonly`
   - `https://www.googleapis.com/auth/calendar.readonly`
2. **User Authorization**: The user selects their Google account and approves the permissions.
3. **Authorization Code**: Google redirects the browser back to our backend callback URL (`/api/auth/google/callback?code=4/0AX4X...`).
4. **Token Exchange**: Our backend sends that temporary code to Google's token endpoint over HTTPS. Google returns:
   - `access_token` : Short-lived credential (valid for 1 hour) used to call Gmail and Calendar APIs.
   - `refresh_token` : Long-lived credential used to request new access tokens automatically when the current one expires.
5. **Database Storage**: The backend stores the tokens securely in the `connected_accounts` table, linked to the user's UUID in the `users` table.

> **Examiner Question**: *"What happens when the 1-hour access token expires?"*
> **Your Answer**: *"Our `GoogleOAuthService` uses the Google OAuth2 client, which automatically uses the stored `refresh_token` to fetch a fresh `access_token` without requiring the user to log in again."*

4.2 How Gmail API Fetches Your Emails

1. The backend calls `gmail.users.messages.list({ userId: 'me', q: 'newer_than:7d', maxResults: 20 })`.
2. This returns a list of message IDs.
3. For each message ID, the backend calls `gmail.users.messages.get({ id, format: 'full' })`.
4. It extracts:
   - Subject (from `headers`)
   - Sender (from `From` header)
   - Date received (from `internalDate`)
   - Body text (by recursively decoding base64-encoded MIME body parts).
5. The raw email is stored in the `emails` table. A unique constraint on `(user_id, message_id)` ensures that existing emails are never duplicated.

4.3 How Google Calendar API Tracks Events

1. The backend calls `calendar.events.list({ calendarId: 'primary', timeMin: (now - 7 days), timeMax: (now + 30 days), singleEvents: true })`.
2. It processes both **exact-time events** (e.g., `start.dateTime: "2026-11-06T14:30:00+05:30"`) and **all-day events** (e.g., `start.date: "2026-11-10"`).
3. Cancelled events with `status: 'cancelled'` are detected and updated in the database so that stale tasks are removed.

4.4 Automatic Background Synchronization

LifeSynch AI does not force users to keep clicking "Refresh".

- `AutoSyncService` initializes when the Express server starts.
- It sets a recurring timer (`setInterval`) that runs every **30,000 milliseconds (30 seconds)**.
- **Safety Mechanism 1 — Concurrency Lock**: A boolean flag `isRunning` prevents overlapping sync cycles if an existing sync takes longer than 30 seconds.
- **Safety Mechanism 2 — Promise.allSettled**: Gmail sync and Calendar sync run in parallel using `Promise.allSettled()`. If the Calendar API encounters a transient network glitch, Gmail sync still completes successfully.
- **Safety Mechanism 3 — Sync State Tracking**: Sync progress and timestamps are written to the `sync_state` table so the frontend status pill (`Synced 1 min ago`) is always up to date.

4.5 AI Responsibility Extraction & Schema Validation

How does raw text become an actionable task?

1. **Prompt Engineering**: The email subject, sender, and body are wrapped in a system prompt that instructs Gemini:
   - Extract actionable responsibilities.
   - Ignore marketing newsletters, spam, and OTPs.

- Separate informational notices into the "Worth Knowing" list.
- Output strict JSON with fields: `title`, `description`, `category`, `priority`, `dueDate`, `dueTime`, `confidenceScore`.

2. **Zod Schema Validation**: Raw LLM output cannot be trusted blindly. The backend parses the JSON through a Zod schema (`z.object({ ... })`). If the LLM invents invalid fields or bad date formats, Zod rejects it immediately.
3. **Database Insertion**: Validated items are saved in the `responsibilities` table with status `pending`.

```
Raw Email/Event  -->  Clean Text  -->  Gemini Prompt  -->  Structured JSON
                                                                  |
Database Storage <-- Validated Record <-- Zod Schema Validation <--+
```

4.6 The "Worth Knowing" Section vs Actionable Tasks

A critical innovation in LifeSynch AI is distinguishing between **actions** and **information**:

- **Actionable Responsibility**: "Submit DBMS Mini Project Report by Thursday 4 PM." (Requires student action; has a strict deadline and priority).
- **Worth Knowing Item**: "The campus canteen will have revised timings during Diwali." (Informational notice; no student action required, but helpful context).

In our database, both live in the `responsibilities` table, but informational items are flagged in the `metadata` JSONB column as `itemType: "worth_knowing"`. The frontend renders them in a dedicated collapsible card, keeping the primary task view clean.

4.7 Deadline & Timezone Intelligence (UTC vs IST)

Timezones are the #1 cause of bugs in calendar and scheduling applications. Here is how LifeSynch AI solves them cleanly:

1. **All-Day Events**: A calendar event on `2026-11-10` has no time component. If converted blindly, UTC offset (-5:30) could shift it to `2026-11-09 18:30:00 UTC`. LifeSynch AI treats all-day dates in local Indian Standard Time (IST, +05:30) and sets the deadline to 23:59:59 IST of that day.
2. **Exact-Time Events**: Parsed with full ISO 8601 offset preservation (`2026-11-10T14:00:00+05:30`).
3. **Due Time Storage**:
   - The date component is stored in `responsibilities.due_date` (`TIMESTAMPTZ`).
   - The specific clock time (e.g., `"14:00"` or `"17:00"`) is preserved in `metadata.dueTime` (`JSONB`).
   - This ensures the UI displays: "*Nov 10, 2026 at 2:00 PM*" without any ambiguity.

4.8 Notifications & Reminders Engine

`ReminderService` scans pending responsibilities and categorizes them into three active reminder buckets:

1. **Due Today**: Items whose deadline falls between `00:00:00` and `23:59:59` of today's local date.
2. **Upcoming**: Items due within the next 48 hours.

3. **Overdue / Expired**: Items whose deadline has already passed.

Completed and Cancelled Exclusion: Responsibilities with status `completed` or `cancelled` are strictly excluded from generating reminders, ensuring users are never alerted for tasks they have already finished.

4.9 "Ask LifeSynch AI" Conversational Assistant

Users can type natural language questions into the assistant widget.

- **Intent Detection**: The query is analyzed for temporal keywords (`"today"` , `"tomorrow"` , `"this week"` , `"next week"`) and category filters (`"academic"` , `"work"`).
- **Dual-Path Architecture**:
  1. *Primary Path*: If live Gemini LLM is active, the assistant uses generative AI with grounded context from the user's database.
  2. *Deterministic Grounded Fallback*: If the Gemini API key has exceeded its daily free-tier quota (HTTP 429), our backend automatically executes an offline rule-based parser that queries the database directly. It answers questions accurately without crashing or hallucinating.
     - This fallback passed 21 out of 21 rigorous test cases in our test suite!

4.10 Multi-User Data Isolation & Row Level Security (RLS)

In a multi-user application, User A must never see User B's emails or responsibilities.

LifeSynch AI enforces multi-layered isolation:

1. **Application Layer**: Every database query in Express controllers specifies `.eq('user_id', req.user.id)` .
2. **Database Layer (RLS)**: PostgreSQL Row Level Security policies check `auth.uid() = user_id` for every `SELECT` , `INSERT` , `UPDATE` , and `DELETE` .
3. Even if a malicious user tampers with client-side API requests, the PostgreSQL database engine blocks access at the kernel level.

---

5. Key Code Walkthroughs with Annotations

Walkthrough A: Automatic Sync Loop (`backend/src/services/sync/auto_sync.service.ts`)

```
class AutoSyncService {
  private syncIntervalMs = 30000; // 30 seconds interval
  private isRunning = false;      // Prevents overlapping cycles

  public start() {
    setInterval(async () => {
      // Step 1: Prevent duplicate concurrent runs
      if (this.isRunning) return;
      this.isRunning = true;

      try {
        // Step 2: Fetch all connected Google accounts
        const accounts = await this.getActiveConnectedAccounts();

        for (const account of accounts) {
          // Step 3: Run Gmail and Calendar concurrently
          await Promise.allSettled([
            gmailSyncService.syncUserEmails(account.userId),
            calendarSyncService.syncUserEvents(account.userId)
          ]);
        }
      } finally {
        // Step 4: Always release the lock, even on error
        this.isRunning = false;
      }
    }, this.syncIntervalMs);
  }
}
```

Walkthrough B: Safe Reminder Resolution (`backend/src/services/reminders/reminder.service.ts`)

```
// Converts any date/time string into a precise Unix timestamp in IST
public resolveDeadlineTimestamp(dueDate: string, dueTime?: string): number {
  if (dueTime) {
    // Exact-time event: combine date and time
    const combined = `${dueDate.split('T')[0]}T${dueTime}:00+05:30`;
    return new Date(combined).getTime();
  }

  // All-day event: set to 23:59:59 IST so user has the whole day
  const dateOnly = dueDate.split('T')[0];
  const endOfDay = `${dateOnly}T23:59:59+05:30`;
  return new Date(endOfDay).getTime();
}
```

6. Common Traps & How We Solved Them

Trap / Challenge	What Usually Goes Wrong	How LifeSynch AI Solved It
Expired Google OAuth Tokens	App crashes with HTTP 401 Unauthorized after 1 hour.	Google OAuth2 client automatically refreshes tokens using the stored `refresh_token`.
LLM Output Hallucination	LLMs return unstructured sentences instead of valid objects.	Strict JSON schema prompt + runtime Zod schema parsing.
API Rate Quota Exhaustion	Live Gemini Free Tier limit (20 req/day) halts the system.	Robust deterministic grounded fallback answers user queries without API dependency.
Timezone Shift (-5:30)	An event on Nov 10 appears as Nov 9 18:30 due to UTC conversion.	Explicit IST offset handling (`+05:30`) and safe date normalization.
Overlapping Sync Cycles	Slow network causes a new sync to fire before the old one finishes.	Boolean `isRunning` guard acts as a mutex lock.

7. Glossary of Key Technical Terms

- **OAuth 2.0**: An open authorization protocol that allows a website or app to access user data from another service (like Google) without giving away the user's password.
- **Access Token**: A temporary digital key used by the backend to make authorized API requests on behalf of the user.
- **Refresh Token**: A long-term digital key used to generate new access tokens when old ones expire.
- **REST API**: Representational State Transfer; an architectural pattern for web services using standard HTTP verbs (`GET` , `POST` , `PUT` , `DELETE`).
- **PostgreSQL**: An open-source relational database that stores data in structured tables with foreign keys and ACID transactions.
- **JSONB**: Binary JSON storage format in PostgreSQL that allows fast indexing and querying of flexible nested data structures.
- **Row Level Security (RLS)**: A PostgreSQL security feature that restricts which table rows a user can query or modify based on security policies.
- **Zod**: A TypeScript-first schema declaration and validation library used to verify data shapes at runtime.
- **Concurrency**: The ability of a computer program to execute multiple operations (e.g., syncing Gmail and Calendar) simultaneously.
- **IST (Indian Standard Time)**: UTC+05:30, the standard timezone used across India.